

数值算法 Homework 13

姓名: 雍崔扬

学号: 21307140051

Due: 08:00 May 26, 2025

Problem 1

Use the finite difference method and the finite element method (with linear elements), both on $n + 1$ equispaced nodes, to solve the boundary value problem

$$\begin{cases} -u''(x) + u(x) = x^2 & (0 < x < 1) \\ u(0) = 0, u(1) = 1 \end{cases}$$

Try a few different values of n and compare your solutions with the exact one.

(1) 精确解

设 $u(x) = u_h(x) + u_p(x)$, 其中:

- 齐次解 $u_h(x) := Ae^x + Be^{-x}$ 满足齐次方程 $-u''(x) + u(x) \equiv 0$
- 特解 $u_p(x) := ax^2 + bx + c$ 为二次多项式

将特解代入非齐次方程 $-u''(x) + u(x) = x^2$ 得:

$$\begin{aligned} -u_p''(x) + u_p(x) &= x^2 \\ \Updownarrow \\ -2a + (ax^2 + bx + c) &= x^2 \end{aligned}$$

比较系数可得 $a = 1, b = 0, c = 2$.

因此特解 $u_p(x) = x^2 + 2$, 通解 $u(x) = u_h(x) + u_p(x) = Ae^x + Be^{-x} + x^2 + 2$

代入边界条件 $u(0) = 0, u(1) = 1$ 可得:

$$\begin{aligned} u(0) &= A + B + 2 = 0 \\ u(1) &= Ae + Be^{-1} + 1 + 2 = 1 \end{aligned}$$

解得 $A = -2/(e + 1), B = -2e/(e + 1)$

因此精确解为:

$$\begin{aligned} u(x) &= Ae^x + Be^{-x} + x^2 + 2 \\ &= -\frac{2}{e + 1}e^x - \frac{2e}{e + 1}e^{-x} + x^2 + 2 \end{aligned}$$

Matlab 代码:

```
A = -2/(exp(1) + 1);  
B = -2*exp(1)/(exp(1) + 1);  
u_exact = @(x) A*exp(x) + B*exp(-x) + x.^2 + 2;
```

(2) FDM

将区间 $[0, 1]$ 划分为 n 段, 节点为:

$$x_i = ih \quad (i = 0, 1, \dots, n)$$

$$h = \frac{1}{n}$$

记 $u_i = u(x_i)$

根据边界条件可知 $u_0 = u(0) = 0$, $u_n = u(1) = 1$

未知的是内部节点处的值 $u_1, \dots, u_{n-1}$.

对于内部节点, 用中心差分逼近二阶导数, 即有:

$$\begin{aligned} -\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + u_i &= x_i^2 \quad (i = 1, \dots, n-1) \\ \Updownarrow \\ -u_{i-1} + (2 + h^2)u_i - u_{i+1} &= h^2 x_i^2 \quad (i = 1, \dots, n-1) \\ \Updownarrow \\ (2 + h^2)u_1 - u_2 &= h^2 x_1^2 + u_0 \\ -u_{i-1} + (2 + h^2)u_i - u_{i+1} &= h^2 x_i^2 \quad (i = 2, \dots, n-2) \\ -u_{n-2} + (2 + h^2)u_{n-1} &= h^2 x_{n-1}^2 + u_n \end{aligned}$$

其矩阵形式为:

$$Au = \begin{bmatrix} 2+h^2 & -1 & & & \\ -1 & 2+h^2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2+h^2 & -1 \\ & & & -1 & 2+h^2 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-2} \\ u_{n-1} \end{bmatrix} = \begin{bmatrix} h^2 x_1^2 \\ h^2 x_2^2 \\ \vdots \\ h^2 x_{n-2}^2 \\ h^2 x_{n-1}^2 + 1 \end{bmatrix} = b$$

求解上述方程即可得到内部节点处的近似值 $u_1, \dots, u_{n-1}$.

Matlab 代码:

```
h = 1/n;
x_nodes = linspace(0,1,n+1)';
N_int = n-1; % 内部未知量 u_1, ..., u_{n-1} 共 n-1 个

% 构造 FDM 系数矩阵: 对角线 (2 + h^2), 次对角线 -1
diag_vals = (2 + h^2) * ones(N_int,1);
off_vals = -1 * ones(N_int,1); % 最后一个元素会被忽略
A_fd = spdiags([off_vals, diag_vals, off_vals], [-1,0,1], N_int, N_int);
b_fd = zeros(N_int,1);

% 构造右端向量 b_fd
for i = 1:N_int
    xi = i*h;
    b_fd(i) = h^2 * xi^2;
end
b_fd(N_int) = b_fd(N_int) + 1; % 边界条件修正: 最后一行加上 u_n = 1 (u_0 = 0 天然成立, 不用调整)
```

% 求解内部节点值

u_fd_int = A_fd \ b_fd; % 列向量, 长度 N_int

u_fd = [0; u_fd_int; 1]; % 加上边界 u(0)=0, u(1)=1

(3) FEM

取等距节点 $a = x_0 < x_1 < \dots < x_n = b$, 其中 $x_i = x_0 + ih$, $h = (b - a)/n$

则一次样条基函数的形式为:

$$\begin{aligned}\phi_0(x) &= \begin{cases} \frac{x_1-x}{h}, & x \in [x_0, x_1] \\ 0, & \text{otherwise} \end{cases} \\ \phi_i(x) &= \begin{cases} \frac{x-x_{i-1}}{h}, & x \in [x_{i-1}, x_i] \\ \frac{x_{i+1}-x}{h}, & x \in (x_i, x_{i+1}] \\ 0, & \text{otherwise} \end{cases} \quad (i = 1, \dots, n-1) \\ \phi_n(x) &= \begin{cases} \frac{x-x_{n-1}}{h}, & x \in [x_{n-1}, x_n] \\ 0, & \text{otherwise} \end{cases}\end{aligned}$$

考虑以下常微分问题:

$$\begin{cases} -u''(x) + u(x) = x^2 & (0 < x < 1) \\ u(0) = 0, u(1) = 1 \end{cases}$$

记 $u(x)$ 在 $\text{span}\{\phi_i(x) : i = 0, 1, \dots, n\}$ 中的投影为 $\hat{u}(x) = \sum_{k=0}^n c_k \phi_k(x)$

令 $\hat{u}(x)$ 满足 Galerkin 条件:

$$\begin{aligned}\langle \hat{u}''(x) - u(x) + x^2, \phi_j(x) \rangle &= \left\langle \sum_{k=0}^n c_k \phi_k''(x) - \sum_{k=0}^n c_k \phi_k(x) + x^2, \phi_j(x) \right\rangle \\ &= \sum_{k=0}^n c_k \langle \phi_k''(x), \phi_j(x) \rangle - \sum_{k=0}^n c_k \langle \phi_k(x), \phi_j(x) \rangle + \langle x^2, \phi_j(x) \rangle \quad (j = 0, 1, \dots, n) \\ &= \sum_{k=0}^n c_k \langle \phi_k'(x), \phi_j'(x) \rangle - \sum_{k=0}^n c_k \langle \phi_k(x), \phi_j(x) \rangle + \langle x^2, \phi_j(x) \rangle \\ &= 0\end{aligned}$$

其中倒数第二步使用了广义导数的定义.

我们定义:

- ① 刚度矩阵 A :

$$\begin{aligned}
A &= \begin{bmatrix} \langle \phi_0''(x), \phi_0(x) \rangle & \langle \phi_1''(x), \phi_0(x) \rangle & \cdots & \langle \phi_{n-1}''(x), \phi_0(x) \rangle & \langle \phi_n''(x), \phi_0(x) \rangle \\ \langle \phi_0''(x), \phi_1(x) \rangle & \langle \phi_1''(x), \phi_1(x) \rangle & \cdots & \langle \phi_{n-1}''(x), \phi_1(x) \rangle & \langle \phi_n''(x), \phi_1(x) \rangle \\ \vdots & \vdots & & \vdots & \vdots \\ \langle \phi_0''(x), \phi_{n-1}(x) \rangle & \langle \phi_1''(x), \phi_{n-1}(x) \rangle & \cdots & \langle \phi_{n-1}''(x), \phi_{n-1}(x) \rangle & \langle \phi_n''(x), \phi_{n-1}(x) \rangle \\ \langle \phi_0''(x), \phi_n(x) \rangle & \langle \phi_1''(x), \phi_n(x) \rangle & \cdots & \langle \phi_{n-1}''(x), \phi_n(x) \rangle & \langle \phi_n''(x), \phi_n(x) \rangle \end{bmatrix} \\
&= \begin{bmatrix} \langle \phi_0'(x), \phi_0'(x) \rangle & \langle \phi_1'(x), \phi_0'(x) \rangle & \cdots & \langle \phi_{n-1}'(x), \phi_0'(x) \rangle & \langle \phi_n'(x), \phi_0'(x) \rangle \\ \langle \phi_0'(x), \phi_1'(x) \rangle & \langle \phi_1'(x), \phi_1'(x) \rangle & \cdots & \langle \phi_{n-1}'(x), \phi_1'(x) \rangle & \langle \phi_n'(x), \phi_1'(x) \rangle \\ \vdots & \vdots & & \vdots & \vdots \\ \langle \phi_0'(x), \phi_{n-1}'(x) \rangle & \langle \phi_1'(x), \phi_{n-1}'(x) \rangle & \cdots & \langle \phi_{n-1}'(x), \phi_{n-1}'(x) \rangle & \langle \phi_n'(x), \phi_{n-1}'(x) \rangle \\ \langle \phi_0'(x), \phi_n'(x) \rangle & \langle \phi_1'(x), \phi_n'(x) \rangle & \cdots & \langle \phi_{n-1}'(x), \phi_n'(x) \rangle & \langle \phi_n'(x), \phi_n'(x) \rangle \end{bmatrix} \\
&= \frac{1}{h} \begin{bmatrix} 1 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & \ddots & \ddots & \\ & & \ddots & 2 & -1 \\ & & & -1 & 1 \end{bmatrix}
\end{aligned}$$

由于只有相邻的基函数的支撑集才相交，故当且仅当 $|j - k| \leq 1$ 时 a_{jk} 才是非零的。

换言之，刚度矩阵 A 是一个三对角阵。

更具体地说，刚度矩阵 A 是对称三对角阵，

因为其元素具有 (共轭) 对称性: $a_{jk} = \langle \phi_k'(x), \phi_j'(x) \rangle = \langle \phi_j'(x), \phi_k'(x) \rangle = a_{kj}$ 。

◦ 考虑对角元 (即 $j = k$ 的情况):

$$\begin{aligned}
a_{jj} &= \langle \phi_j'(x), \phi_j'(x) \rangle \\
&= \begin{cases} \int_{x_0}^{x_1} \left(-\frac{1}{h}\right)^2 dx, & j = 0 \\ \int_{x_{j-1}}^{x_j} \left(\frac{1}{h}\right)^2 dx + \int_{x_j}^{x_{j+1}} \left(-\frac{1}{h}\right)^2 dx, & j = 1, 2, \dots, n-1 \\ \int_{x_{n-1}}^{x_n} \left(\frac{1}{h}\right)^2 dx, & j = n \end{cases} \\
&= \begin{cases} \frac{1}{h}, & j = 0 \\ \frac{2}{h}, & j = 1, 2, \dots, n-1 \\ \frac{1}{h}, & j = n \end{cases}
\end{aligned}$$

◦ 考虑次对角元和超对角元 (即 $|j - k| = 1$ 的情况):

$$\begin{aligned}
a_{j,j+1} &= \langle \phi_{j+1}'(x), \phi_j'(x) \rangle \\
&= \int_{x_j}^{x_{j+1}} \frac{1}{h} \cdot \left(-\frac{1}{h}\right) dx \\
&= -\frac{1}{h^2} \cdot h \\
&= -\frac{1}{h}
\end{aligned}$$

Since A is symmetric, we have $a_{j+1,j} = a_{j,j+1} = -\frac{1}{h}$

- ② 质量矩阵 M :

$$\begin{aligned}
M &= \begin{bmatrix} \langle \phi_0(x), \phi_0(x) \rangle & \langle \phi_1(x), \phi_0(x) \rangle & \cdots & \langle \phi_{n-1}(x), \phi_0(x) \rangle & \langle \phi_n(x), \phi_0(x) \rangle \\ \langle \phi_0(x), \phi_1(x) \rangle & \langle \phi_1(x), \phi_1(x) \rangle & \cdots & \langle \phi_{n-1}(x), \phi_1(x) \rangle & \langle \phi_n(x), \phi_1(x) \rangle \\ \vdots & \vdots & & \vdots & \vdots \\ \langle \phi_0(x), \phi_{n-1}(x) \rangle & \langle \phi_1(x), \phi_{n-1}(x) \rangle & \cdots & \langle \phi_{n-1}(x), \phi_{n-1}(x) \rangle & \langle \phi_n(x), \phi_{n-1}(x) \rangle \\ \langle \phi_0(x), \phi_n(x) \rangle & \langle \phi_1(x), \phi_n(x) \rangle & \cdots & \langle \phi_{n-1}(x), \phi_n(x) \rangle & \langle \phi_n(x), \phi_n(x) \rangle \end{bmatrix} \\
&= \frac{h}{6} \begin{bmatrix} 2 & 1 & & & \\ 1 & 4 & 1 & & \\ & 1 & \ddots & \ddots & \\ & & \ddots & 4 & 1 \\ & & & 1 & 2 \end{bmatrix}
\end{aligned}$$

由于只有相邻的基函数的支撑集才相交，故当且仅当 $|j - k| \leq 1$ 时 m_{jk} 才是非零的。

换言之，质量矩阵 M 是一个三对角阵。

更具体地来说，质量矩阵 M 是对称三对角阵，

因为其元素具有 (共轭) 对称性: $m_{jk} = \langle \phi_k(x), \phi_j(x) \rangle = \langle \phi_j(x), \phi_k(x) \rangle = m_{kj}$ 。

- 考虑对角元 (即 $j = k$ 的情况):

$$\begin{aligned}
m_{jj} &= \langle \phi_j(x), \phi_j(x) \rangle \\
&= \begin{cases} \int_{x_0}^{x_1} \left(\frac{x_0 - x}{h}\right)^2 dx, & j = 0 \\ \int_{x_{j-1}}^{x_j} \left(\frac{x - x_{j-1}}{h}\right)^2 dx + \int_{x_j}^{x_{j+1}} \left(\frac{x_j - x}{h}\right)^2 dx, & j = 1, 2, \dots, n-1 \\ \int_{x_{n-1}}^{x_n} \left(\frac{1}{h}\right)^2 dx, & j = n \end{cases} \\
&= \begin{cases} \frac{1}{h^2} \int_0^h x^2 dx, & j = 0 \\ \frac{2}{h^2} \int_0^h x^2 dx, & j = 1, 2, \dots, n-1 \\ \frac{1}{h^2} \int_0^h x^2 dx, & j = n \end{cases} \\
&= \begin{cases} \frac{h}{3}, & j = 0 \\ \frac{2h}{3}, & j = 1, 2, \dots, n-1 \\ \frac{h}{3}, & j = n \end{cases}
\end{aligned}$$

- 考虑次对角元和超对角元 (即 $|j - k| = 1$ 的情况):

$$\begin{aligned}
m_{j,j+1} &= \langle \phi_{j+1}(x), \phi_j(x) \rangle \\
&= \int_{x_j}^{x_{j+1}} \frac{x - x_j}{h} \cdot \frac{x_{j+1} - x}{h} dx \\
&= -\frac{1}{h^2} \int_0^h x(h - x) dx \\
&= \frac{h}{6}
\end{aligned}$$

Since M is symmetric, we have $m_{j+1,j} = m_{j,j+1} = \frac{h}{6}$

•

- ③ 载荷向量 b :

$$b = \begin{bmatrix} \langle x^2, \phi_0(x) \rangle \\ \langle x^2, \phi_1(x) \rangle \\ \vdots \\ \langle x^2, \phi_j(x) \rangle \\ \vdots \\ \langle x^2, \phi_{n-1}(x) \rangle \\ \langle x^2, \phi_n(x) \rangle \end{bmatrix} = \frac{h^3}{12} \begin{bmatrix} 1 \\ 12 + 2 \\ \vdots \\ 12j^2 + 2 \\ \vdots \\ 12(n-1)^2 + 2 \\ 6n^2 - 4n + 1 \end{bmatrix}$$

考虑 b 的第 j 个元素:

$$\begin{aligned} b_j &= \langle x^2, \phi_j(x) \rangle \\ &= \int_0^1 x^2 \phi_j(x) dx \\ &= \begin{cases} \int_{x_0}^{x_1} x^2 \cdot \frac{x_0-x}{h} dx, & j=0 \\ \int_{x_{j-1}}^{x_j} x^2 \cdot \frac{x-x_{j-1}}{h} dx + \int_{x_j}^{x_{j+1}} x^2 \cdot \frac{x_j-x}{h} dx, & j=1, \dots, n-1 \\ \int_{x_{n-1}}^{x_n} x^2 \cdot \frac{x-x_{n-1}}{h} dx, & j=n \end{cases} \\ &= \begin{cases} \frac{h^3}{12}, & j=0 \\ \frac{h^3}{12}(12j^2+2), & j=1, \dots, n-1 \\ \frac{h^3}{12}(6n^2-4n+1), & j=n \end{cases} \end{aligned}$$

注意到边界条件为 $u(0) = 0, u(1) = 1$,

故可设 $c_0 = 0, c_n = 1, f_0 = 0, f_n = n$,

并将系数矩阵 $A = M$ 的第 0 行置为 $[1, 0, \dots, 0]$, 将第 n 行置为 $[0, \dots, 0, 1]$.

求解对应方程即得未知的基函数系数 $c_1, \dots, c_{n-1}$.

值得注意的是, $x_0, x_1, \dots, x_n$ 并不一定要是等距节点,

不过形式会更复杂一些 (参考代码的实现, 它使用了单元刚度矩阵和单元质量矩阵).

Matlab 代码:

```
h = 1/n;
x_nodes = linspace(0,1,n+1)';

% --- 构造全局刚度矩阵 K 和质量矩阵 M ---
K = sparse(n+1, n+1);
M = sparse(n+1, n+1);
for k = 1:n
    xL = x_nodes(k);
    xR = x_nodes(k+1);
    he = xR - xL; % 单元长度 = h

    % 单元刚度矩阵 ke = (1/he)*[1 -1; -1 1]
    ke = (1/he) * [1, -1; -1, 1];
    % 单元质量矩阵 me = (he/6)*[2 1; 1 2]
    me = (he/6) * [2, 1; 1, 2];

    % 将单元刚度和质量组装到全局矩阵
    idxs = [k, k+1]; % 全局节点索引
    K(idxs,idxs) = K(idxs,idxs) + ke;
```

```

M(idxs,idxs) = M(idxs,idxs) + Me;
end

% 全局矩阵 A_fem = K + M (对应 -u'' + u 的弱形式)
A_fem = K + M;

% --- 构造载荷向量 f (有解析公式, 无需使用数值积分公式) ---
f = zeros(n+1,1);

% j=0 对应 MATLAB 下标 1
f(1) = (h^3/12) * 1;

% j = 1,2,...,n-1 对应 MATLAB 下标 2:(n)
for j = 1:n-1
f(j+1) = (h^3/12) * (12*j^2 + 2);
end

% j = n 对应 MATLAB 下标 n+1
f(n+1) = (h^3/12) * (6*n^2 - 4*n + 1);

% --- 强制 Dirichlet 边界条件 u(0)=0, u(1)=1 ---
A_fem(1,:) = 0;    A_fem(1,1) = 1;    f(1) = 0;
A_fem(n+1,:) = 0;  A_fem(n+1,n+1) = 1;  f(n+1) = 1;

% 求解 FEM 系统
u_fem = A_fem \ f;    % 列向量, 长度 n+1

```

(4) 运行结果

考虑不同 n 下 FDM 和 FEM 在节点上误差的最大值:

n	FDM	FEM
2	4.1400e-03	4.0069e-04
4	1.0589e-03	1.0892e-04
8	2.6628e-04	2.7761e-05
16	6.6669e-05	6.9731e-06

对比可知 FEM 有着更高的精度.

Problem 2

Use the finite difference method and the finite element method (with linear elements), both on $n + 1$ equispaced nodes, to solve the eigenvalue problem

$$\begin{cases} -u''(x) = \lambda u(x) & (0 < x < \pi) \\ u(0) = 0, u(\pi) = 0 \end{cases}$$

Compute a few smallest eigenvalues and the corresponding eigenfunctions. Compare your solutions with the exact ones.

(1) 精确解

容易看出特征对为:

$$\begin{cases} \lambda_k = k^2 \\ u_k(x) = \sin(kx) \end{cases} \quad (k = 1, 2, \dots)$$

Matlab 代码为:

```
% 解析本征值  $\lambda_k = k^2$ , 解析特征函数  $u_k(x) = \sin(kx)$ 
exact_lambda = @(k) k^2;
exact_uk = @(k, x) sin(k * x);
```

(2) FDM

将区间 $[0, \pi]$ 划分为 n 段, 节点为:

$$x_i = ih \quad (i = 0, 1, \dots, n)$$
$$h = \frac{\pi}{n}$$

记 $u_i = u(x_i)$

根据边界条件可知 $u_0 = u(0) = 0$, $u_n = u(\pi) = 0$

未知的是内部节点处的值 $u_1, \dots, u_{n-1}$.

对于内部节点, 用中心差分逼近二阶导数, 即有:

$$-\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} = \lambda u_i \quad (i = 1, \dots, n-1)$$
$$\Updownarrow$$
$$-u_{i-1} + 2u_i - u_{i+1} = \lambda(h^2 u_i) \quad (i = 1, \dots, n-1)$$

代入 $u_0 = u_n = 0$ 可将上述方程整理成矩阵形式:

$$Au = \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-2} \\ u_{n-1} \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-2} \\ u_{n-1} \end{bmatrix} \quad \lambda = u \cdot \lambda h^2$$

因此我们只取求解特征值问题 $Au = u \cdot \mu$, 再应用 $\lambda = \frac{\mu}{h^2}$ 即可解得特征值.

Matlab 代码:

```
% 内部未知量  $u_1, \dots, u_{n-1}$ , 共  $n-1$  个
N_int = n - 1;

% 构造三对角矩阵 T, 大小  $(n-1) \times (n-1)$ ,
% 对角线全部是 2, 次对角线 -1
e = ones(N_int, 1);
```

```

T = spdiags([-e, 2*e, -e], [-1,0,1], N_int, N_int);

% FDM 本征问题: T * u_int = μ * u_int, μ = λ * h^2
% 用 eigs 求前 num_eigs 小特征值 / 特征向量
[u_fd_int, D_fd] = eigs(T, num_eigs, 'SM');

mu_fd = diag(D_fd);           % T 的本征值 μ_j
lambda_fd = mu_fd / (h^2);    % 对应的 λ_j ≈ μ_j / h^2

% 将内部解扩展到全节点 (包括 0 和 π 处)
u_fd = zeros(n+1, num_eigs); % 行数 n+1, 对应节点 0..n
for j = 1:num_eigs
    u_int = u_fd_int(:,j);
    if u_int(1) < 0 % 如果 u 的符号不对, 则取负
        u_int = -u_int;
    end
    % 内部节点 u_1..u_{n-1} 存在 u_fd_int(:,j)
    u_temp = [0; u_int; 0]; % 在头尾补 0
    % 归一化: 使数值解在节点上的最大绝对值为 1
    u_temp = u_temp / max(abs(u_temp));
    u_fd(:,j) = u_temp;
end

```

(3) FEM

取等距节点 $a = x_0 < x_1 < \cdots < x_n = b$, 其中 $x_i = x_0 + ih$, $h = (b - a)/n$
 则一次样条基函数的形式为:

$$\begin{aligned}
 \phi_0(x) &= \begin{cases} \frac{x_1-x}{h}, & x \in [x_0, x_1] \\ 0, & \text{otherwise} \end{cases} \\
 \phi_i(x) &= \begin{cases} \frac{x-x_{i-1}}{h}, & x \in [x_{i-1}, x_i] \\ \frac{x_{i+1}-x}{h}, & x \in (x_i, x_{i+1}] \\ 0, & \text{otherwise} \end{cases} \quad (i = 1, \dots, n-1) \\
 \phi_n(x) &= \begin{cases} \frac{x-x_{n-1}}{h}, & x \in [x_{n-1}, x_n] \\ 0, & \text{otherwise} \end{cases}
 \end{aligned}$$

考虑以下特征值问题:

$$\begin{cases} -u''(x) = \lambda u(x) & (0 < x < \pi) \\ u(0) = 0, u(\pi) = 0 \end{cases}$$

记 $u(x)$ 在 $\text{span}\{\phi_i(x) : i = 0, 1, \dots, n\}$ 中的投影为 $\hat{u}(x) = \sum_{k=0}^n c_k \phi_k(x)$
 根据边界条件 $u(0) = u(\pi) = 0$ 可知 $c_0 = c_n = 0$, 因此 $\hat{u}(x) = \sum_{k=1}^{n-1} c_k \phi_k(x)$
 令 $\hat{u}(x)$ 满足 Galerkin 条件:

$$\begin{aligned}
\langle \hat{u}''(x) + \lambda u(x), \phi_j(x) \rangle &= \left\langle \sum_{k=1}^{n-1} c_k \phi_k''(x) + \lambda \sum_{k=1}^{n-1} c_k \phi_k(x), \phi_j(x) \right\rangle \\
&= \sum_{k=1}^{n-1} c_k \langle \phi_k''(x), \phi_j(x) \rangle + \lambda \sum_{k=1}^{n-1} c_k \langle \phi_k(x), \phi_j(x) \rangle \quad (j = 1, \dots, n-1) \\
&= \sum_{k=1}^{n-1} c_k \langle \phi_k'(x), \phi_j'(x) \rangle + \lambda \sum_{k=1}^{n-1} c_k \langle \phi_k(x), \phi_j(x) \rangle \\
&= 0
\end{aligned}$$

其中倒数第二步使用了广义导数的定义.

我们定义:

- ① 刚度矩阵 A :

$$\begin{aligned}
A &= \begin{bmatrix} \langle \phi_1''(x), \phi_1(x) \rangle & \langle \phi_2''(x), \phi_1(x) \rangle & \cdots & \langle \phi_{n-1}''(x), \phi_1(x) \rangle \\ \langle \phi_1''(x), \phi_2(x) \rangle & \langle \phi_2''(x), \phi_2(x) \rangle & \cdots & \langle \phi_{n-1}''(x), \phi_2(x) \rangle \\ \vdots & \vdots & & \vdots \\ \langle \phi_1''(x), \phi_{n-1}(x) \rangle & \langle \phi_2''(x), \phi_{n-1}(x) \rangle & \cdots & \langle \phi_{n-1}''(x), \phi_{n-1}(x) \rangle \end{bmatrix} \\
&= \begin{bmatrix} \langle \phi_1'(x), \phi_1'(x) \rangle & \langle \phi_2'(x), \phi_1'(x) \rangle & \cdots & \langle \phi_{n-1}'(x), \phi_1'(x) \rangle \\ \langle \phi_1'(x), \phi_2'(x) \rangle & \langle \phi_2'(x), \phi_2'(x) \rangle & \cdots & \langle \phi_{n-1}'(x), \phi_2'(x) \rangle \\ \vdots & \vdots & & \vdots \\ \langle \phi_1'(x), \phi_{n-1}'(x) \rangle & \langle \phi_2'(x), \phi_{n-1}'(x) \rangle & \cdots & \langle \phi_{n-1}'(x), \phi_{n-1}'(x) \rangle \end{bmatrix} \\
&= \frac{1}{h} \begin{bmatrix} 2 & -1 & & \\ -1 & 2 & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 2 \end{bmatrix}
\end{aligned}$$

由于只有相邻的基函数的支撑集才相交, 故当且仅当 $|j - k| \leq 1$ 时 a_{jk} 才是非零的.

换言之, 刚度矩阵 A 是一个三对角阵.

更具体地来说, 刚度矩阵 A 是对称三对角阵,

因为其元素具有 (共轭) 对称性: $a_{jk} = \langle \phi_k'(x), \phi_j'(x) \rangle = \langle \phi_j'(x), \phi_k'(x) \rangle = a_{kj}$.

- 考虑对角元 (即 $j = k$ 的情况):

$$\begin{aligned}
a_{jj} &= \langle \phi_j'(x), \phi_j'(x) \rangle \\
&= \int_{x_{j-1}}^{x_j} \left(\frac{1}{h} \right)^2 dx + \int_{x_j}^{x_{j+1}} \left(-\frac{1}{h} \right)^2 dx \quad (j = 1, \dots, n-1) \\
&= \frac{2}{h}
\end{aligned}$$

- 考虑次对角元和超对角元 (即 $|j - k| = 1$ 的情况):

$$\begin{aligned}
a_{j,j+1} &= \langle \phi'_{j+1}(x), \phi'_j(x) \rangle \\
&= \int_{x_j}^{x_{j+1}} \frac{1}{h} \cdot \left(-\frac{1}{h}\right) dx \\
&= -\frac{1}{h^2} \cdot h \\
&= -\frac{1}{h}
\end{aligned}$$

Since A is symmetric, we have $a_{j+1,j} = a_{j,j+1} = -\frac{1}{h}$

- ② 质量矩阵 M :

$$\begin{aligned}
M &= \begin{bmatrix} \langle \phi_1(x), \phi_1(x) \rangle & \langle \phi_2(x), \phi_1(x) \rangle & \cdots & \langle \phi_{n-1}(x), \phi_1(x) \rangle \\ \langle \phi_1(x), \phi_2(x) \rangle & \langle \phi_2(x), \phi_2(x) \rangle & \cdots & \langle \phi_{n-1}(x), \phi_2(x) \rangle \\ \vdots & \vdots & & \vdots \\ \langle \phi_1(x), \phi_{n-1}(x) \rangle & \langle \phi_2(x), \phi_{n-1}(x) \rangle & \cdots & \langle \phi_{n-1}(x), \phi_{n-1}(x) \rangle \end{bmatrix} \\
&= \frac{h}{6} \begin{bmatrix} 4 & 1 & & \\ 1 & 4 & \ddots & \\ & \ddots & \ddots & 1 \\ & & 1 & 4 \end{bmatrix}
\end{aligned}$$

由于只有相邻的基函数的支撑集才相交，故当且仅当 $|j - k| \leq 1$ 时 m_{jk} 才是非零的。
换言之，质量矩阵 M 是一个三对角阵。

更具体地说，质量矩阵 M 是对称三对角阵，

因为其元素具有 (共轭) 对称性: $m_{jk} = \langle \phi_k(x), \phi_j(x) \rangle = \langle \phi_j(x), \phi_k(x) \rangle = m_{kj}$ 。

- 考虑对角元 (即 $j = k$ 的情况):

$$\begin{aligned}
m_{jj} &= \langle \phi_j(x), \phi_j(x) \rangle \\
&= \int_{x_{j-1}}^{x_j} \left(\frac{x - x_{j-1}}{h}\right)^2 dx + \int_{x_j}^{x_{j+1}} \left(\frac{x_j - x}{h}\right)^2 dx \\
&= \frac{2}{h^2} \int_0^h x^2 dx \\
&= \frac{2h}{3}
\end{aligned}$$

- 考虑次对角元和超对角元 (即 $|j - k| = 1$ 的情况):

$$\begin{aligned}
m_{j,j+1} &= \langle \phi_{j+1}(x), \phi_j(x) \rangle \\
&= \int_{x_j}^{x_{j+1}} \frac{x - x_j}{h} \cdot \frac{x_{j+1} - x}{h} dx \\
&= -\frac{1}{h^2} \int_0^h x(h - x) dx \\
&= \frac{h}{6}
\end{aligned}$$

Since M is symmetric, we have $m_{j+1,j} = m_{j,j+1} = \frac{h}{6}$

最终我们需要求解如下广义特征值问题:

$$Ac = Mc \cdot \lambda$$

Matlab 代码如下:

```
% 内部未知量  $u_1, \dots, u_{n-1}$ , 共  $n-1$  个
N_int = n - 1;

% 构造刚度矩阵  $K$  和质量矩阵  $M$  (大小  $(n+1) \times (n+1)$ )
K_full = sparse(n+1, n+1);
M_full = sparse(n+1, n+1);

% 单元刚度  $K_e$  和质量  $M_e$ 
% 在每个单元  $[x_{i-1}, x_i]$  上, 线性基函数导数为  $\pm 1/h$ 
%  $K_e = (1/h) * [1 \ -1; -1 \ 1]$ 
%  $M_e = (h/6) * [4 \ 1; 1 \ 4]$ 
for i = 1:n
    % 单元索引对应全局节点 (i, i+1)
    idx = [i, i+1];

    % 局部刚度矩阵
    Ke = (1/h) * [1, -1; -1, 1];
    % 局部质量矩阵
    Me = (h/6) * [4, 1; 1, 4];

    % 将  $K_e$  和  $M_e$  装配到全局
    K_full(idx, idx) = K_full(idx, idx) + Ke;
    M_full(idx, idx) = M_full(idx, idx) + Me;
end

% 去掉第 1 行/列和第 n+1 行/列 (对应 Dirichlet 边界), 得到  $(n-1) \times (n-1)$  的  $K$  和  $M$ 
idx_int = 2:n; % 内部节点索引
K = K_full(idx_int, idx_int);
M = M_full(idx_int, idx_int);

% FEM 广义本征问题:  $K c = \lambda M c$ 
% 用 eigs 求前 num_eigs 小  $\lambda$ 
[U_fem_int, D_fem] = eigs(K, M, num_eigs, 'SM');

lambda_fem = diag(D_fem); % FEM 得到的  $\lambda_j$ 

% 将 FEM 的系数向量扩展到全节点 (包括边界  $c_0=c_n=0$ )
U_fem = zeros(n+1, num_eigs);
for j = 1:num_eigs
    c_int = U_fem_int(:, j);
    if c_int(1) < 0 % 如果  $c$  的符号不对, 则取负
        c_int = -c_int;
    end
    u_temp = [0; c_int; 0]; % 在两端补 0
    % 对数值特征向量进行归一化 (使最大绝对值为 1)
    u_temp = u_temp / max(abs(u_temp));
    U_fem(:, j) = u_temp;
end
```

(4) 运行结果

Matlab 代码:

```
% 输出与比较
fprintf('\n k      Exact  $\lambda_k$       FDM  $\lambda$       RelErr(FDM)      FEM  $\lambda$       RelErr(FEM)\n');
for j = 1:num_eigs
    k_exact = j;
    lam_ex = exact_lambda(k_exact);
    lam_fd = lambda_fd(j);
    lam_fem = lambda_fem(j);
    err_fd = abs(lam_fd - lam_ex) / lam_ex;
    err_fem = abs(lam_fem - lam_ex) / lam_ex;
    fprintf('%3d      %10.6f      %10.6f      %10.2e      %10.6f      %10.2e\n', ...
        k_exact, lam_ex, lam_fd, err_fd, lam_fem, err_fem);
end

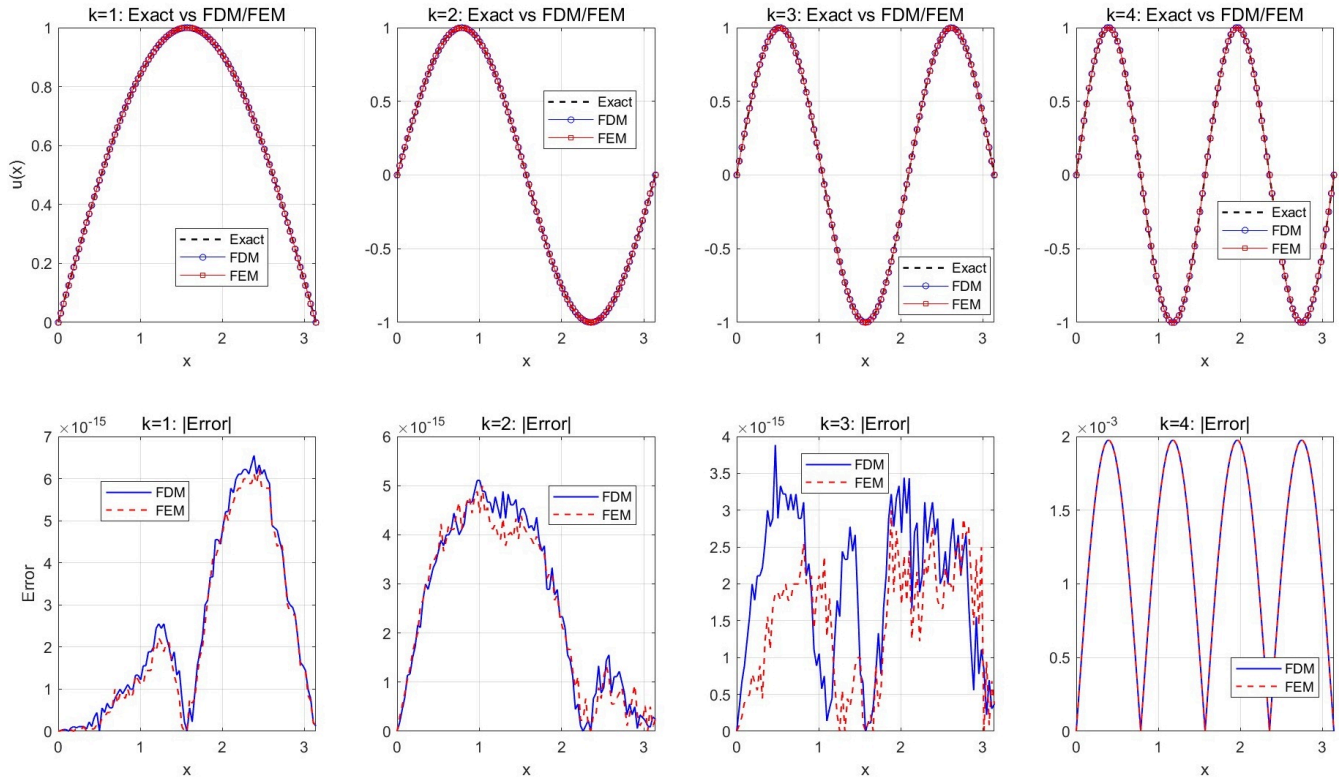
% 可视化: 比较前四个特征函数
figure('Units','normalized','Position',[0.1,0.1,0.8,0.6]);
for j = 1:num_eigs
    subplot(2, num_eigs, j);
    plot(x_nodes, exact_uk(j, x_nodes), 'k--', 'Linewidth',1.2);
    hold on;
    plot(x_nodes, U_fd(:, j), 'b-o', 'MarkerSize',4, 'DisplayName','FDM');
    plot(x_nodes, U_fem(:, j), 'r-s', 'MarkerSize',4, 'DisplayName','FEM');
    hold off;
    title(sprintf('k=%d: Exact vs FDM/FEM', j), 'FontSize', 12);
    if j==1, ylabel('u(x)'); end
    xlabel('x');
    legend({'Exact','FDM','FEM'}, 'Location','Best');
    grid on;

    % 误差曲线
    subplot(2, num_eigs, num_eigs + j);
    plot(x_nodes, abs(U_fd(:,j) - exact_uk(j, x_nodes)), 'b-', 'Linewidth',1);
    hold on;
    plot(x_nodes, abs(U_fem(:,j) - exact_uk(j, x_nodes)), 'r--', 'Linewidth',1);
    hold off;
    title(sprintf('k=%d: |Error|', j), 'FontSize', 12);
    if j==1, ylabel('Error'); end
    xlabel('x');
    legend({'FDM','FEM'}, 'Location','Best');
    grid on;
end
sgtitle(sprintf('FDM vs FEM Eigenfunctions (n = %d, first %d modes)', n, num_eigs),
    'FontSize', 14);
```

运行结果:

k	Exact λ_k	FDM λ	RelErr(FDM)	FEM λ	RelErr(FEM)
1	1.000000	0.999918	8.22e-05	0.600010	4.00e-01
2	4.000000	3.998684	3.29e-04	2.400158	4.00e-01
3	9.000000	8.993340	7.40e-04	5.400798	4.00e-01
4	16.000000	15.978956	1.32e-03	9.602517	4.00e-01

FDM vs FEM Eigenfunctions (n = 100, first 4 modes)



Problem 3

Solve the partial differential equation

$$\begin{cases} \frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2} = 0, & (-1 < x < 1, -1 < y < 1) \\ u(x, -1) = u(x, 1) = x + 1, & (-1 \leq x \leq 1) \\ u(-1, y) = y^2 - 1, u(1, y) = y^2 + 1, & (-1 < y < 1) \end{cases}$$

Visualize your solution.

(1) Jacobi 迭代

```
% Jacobi Iteration to solve 2D Poisson Problem
function [U, errorHistory] = Poisson_Jacobi(U, F, maxIter, tolerance)

n = size(U, 1) - 2; % Get the grid size excluding the boundary points (n x n grid)
h = 1 / (n + 1); % Grid spacing
h_square_times_F = h^2 * F; % Precompute h^2 * F for efficiency in the Jacobi update

% Initialize error history array
errorHistory = zeros(maxIter, 1);

for iter = 1:maxIter
```

```

        % Update interior points using the Jacobi method
        U_new = 0.25 * (U(1:n,2:n+1) + U(3:n+2,2:n+1) + U(2:n+1,1:n) + U(2:n+1,3:n+2) +
h_square_times_F);

        % Calculate the error (difference between new and old solution)
        error = max(max(abs(U_new - U(2:n+1, 2:n+1)))));

        % Store the error in the error history array
        errorHistory(iter) = error;

        % Update U for the next iteration
        U(2:n+1, 2:n+1) = U_new;

        % Check for convergence
        if error < tolerance
            fprintf('Jacobi: Convergence reached after %d iterations.\n', iter);
            errorHistory = errorHistory(1:iter); % Trim the error history to the actual
number of iterations
            break;
        end
    end
end

```

(2) Gauss-Seidel 迭代

```

% Red-Black Gauss-Seidel Iteration to solve 2D Poisson Problem
function [U, errorHistory] = Poisson_RedBlack_GaussSeidel(U, F, maxIter, tolerance)

    n = size(U, 1) - 2; % Get the grid size excluding the boundary points (n x n grid)
    h = 1 / (n + 1); % Grid spacing
    h_square_times_F = h^2 * F; % Precompute h^2 * F for efficiency in the Gauss-Seidel
update

    % Initialize error history array
    errorHistory = zeros(maxIter, 1); % Store the error at each iteration

    for iter = 1:maxIter
        % Save the old version for convergence check
        U_old = U;

        % Update red nodes (i + j is even)
        for i = 2:n+1
            for j = 2:n+1
                if mod(i + j, 2) == 0 % Red node (i+j is even)
                    U(i,j) = 0.25 * (U(i-1,j) + U(i+1,j) + U(i,j-1) + U(i,j+1) +
h_square_times_F(i-1,j-1));
                end
            end
        end
    end
end

```

```

% Update black nodes (i + j is odd)
for i = 2:n+1
    for j = 2:n+1
        if mod(i + j, 2) == 1 % Black node (i+j is odd)
            U(i,j) = 0.25 * (U(i-1,j) + U(i+1,j) + U(i,j-1) + U(i,j+1) +
h_square_times_F(i-1,j-1));
        end
    end
end

% Calculate the error (difference between new and old solution)
error = max(max(abs(U(2:n+1, 2:n+1) - U_old(2:n+1, 2:n+1))));

% Store the error in the error history array for convergence tracking
errorHistory(iter) = error;

% Check for convergence: If error is below tolerance, break the loop
if error < tolerance
    fprintf('Gauss-Seidel: Convergence reached after %d iterations.\n', iter); %
Print convergence message
    errorHistory = errorHistory(1:iter); % Trim the error history to the actual
number of iterations
    break; % Exit the loop if convergence is reached
end
end
end

```

(3) 离散化

使用有限差分去逼近所给 Laplace 方程可得:

$$\frac{-u_{i-1,j} + 2u_{ij} - u_{i+1,j}}{h^2} + \frac{-u_{i,j-1} + 2u_{ij} - u_{i,j+1}}{h^2} = 0 \quad (1 \leq i, j \leq n)$$

$$\text{where } h := \frac{2}{n+1} \text{ and } u_{i,j} := u(x_i, y_j) = u(-1 + ih, -1 + jh)$$

$$\Updownarrow$$

$$4u_{ij} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} = 0 \quad (1 \leq i, j \leq n)$$

$$\Updownarrow$$

$$T_n U + U T_n = B$$

$$T_n = \begin{bmatrix} 2 & -1 & & \\ -1 & \ddots & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 2 \end{bmatrix} \quad U = \begin{bmatrix} u_{1,1} & u_{1,2} & \cdots & u_{1,n} \\ u_{2,1} & u_{2,2} & \cdots & u_{2,n} \\ \vdots & \vdots & & \vdots \\ u_{n,1} & u_{n,2} & \cdots & u_{n,n} \end{bmatrix}$$

$$B = \begin{bmatrix} u_{1,0} + u_{0,1} & u_{0,2} & \cdots & u_{0,n-1} & u_{0,n} + u_{1,n+1} \\ u_{2,0} & 0 & \cdots & 0 & u_{2,n+1} \\ \vdots & \vdots & & \vdots & \vdots \\ u_{n-1,0} & 0 & \cdots & 0 & u_{n-1,n+1} \\ u_{n,0} + u_{n+1,1} & u_{n+1,2} & \cdots & u_{n+1,n-1} & u_{n+1,n} + u_{n,n+1} \end{bmatrix}$$

$$\Updownarrow$$

$$T_{n \times n} \text{vec}(U) = \text{vec}(B)$$

$$\text{where } T_{n \times n} := (I_n \otimes T_n + T_n \otimes I_n) = \begin{bmatrix} T_n & & & \\ & \ddots & & \\ & & \ddots & \\ & & & T_n \end{bmatrix} + \begin{bmatrix} 2I_n & -I_n & & \\ -I_n & \ddots & \ddots & \\ & \ddots & \ddots & -I_n \\ & & -I_n & 2I_n \end{bmatrix}$$

Matlab 代码:

```
% Parameters
n = 50; % Number of grid points in each direction
maxIter = 5000; % Maximum number of iterations
tolerance = 1e-6; % Convergence tolerance

% Grid spacing
h = 2 / (n + 1);

% The source term (right-hand side of Poisson equation)
F = zeros(n, n);

% Create the grid (including boundary)
U = zeros(n+2, n+2); % Solution grid (with boundaries included)

% Set initial condition at y = 0
x = linspace(-1, 1, n+2); % x grid points, including boundary points
y = linspace(-1, 1, n+2); % y grid points, including boundary points
U(1, :) = x + 1; % Set u(x, -1) = x + 1
U(n+2, :) = x + 1; % Set u(x, 1) = x + 1
U(:, 1) = (y.^2 - 1)'; % Set u(-1, y) = y^2 - 1
```

```

U(:, n+2) = (y.^2 + 1)'; % Set u(1, y) = y^2 + 1

% Use Jacobi iteration to solve 2D Poisson problem
[U_Jacobi, errorHistory_Jacobi] = Poisson_Jacobi(U, F, maxIter, tolerance);

% Visualization of the solution for Jacobi method
figure;
[X, Y] = meshgrid(-1:h:1, -1:h:1); % Create meshgrid for plotting
surf(X, Y, U_Jacobi); % Plot the solution
title('Solution of the 2D Laplace Equation (Jacobi)');
xlabel('x');
ylabel('y');
zlabel('u(x, y)');
colorbar;

% Use Gauss-Seidel iteration to solve 2D Poisson problem
[U_GS, errorHistory_GS] = Poisson_RedBlack_GaussSeidel(U, F, maxIter, tolerance);

% Visualization of the solution for Gauss-Seidel method
figure;
[X, Y] = meshgrid(-1:h:1, -1:h:1); % Create meshgrid for plotting
surf(X, Y, U_GS); % Plot the solution
title('Solution of the 2D Laplace Equation (Gauss-Seidel)');
xlabel('x');
ylabel('y');
zlabel('u(x, y)');
colorbar;

% Plot the error history
figure;
plot(1:length(errorHistory_Jacobi), log10(errorHistory_Jacobi), 'r-', 'Linewidth', 1.5);
hold on;
plot(1:length(errorHistory_GS), log10(errorHistory_GS), 'b-', 'Linewidth', 1.5);

% Plot the tolerance line
tolerance_line = log10(tolerance);
plot([1, max(length(errorHistory_Jacobi), length(errorHistory_GS)) + 100], [tolerance_line,
tolerance_line], ...
      'k--', 'Linewidth', 1);
xlabel('Iteration');
ylabel('log10(Error)');
title('Convergence History for Jacobi and Gauss-Seidel Methods');
legend({'Jacobi', 'Gauss-Seidel', 'Tolerance'}, 'Location', 'northeast');
grid on;

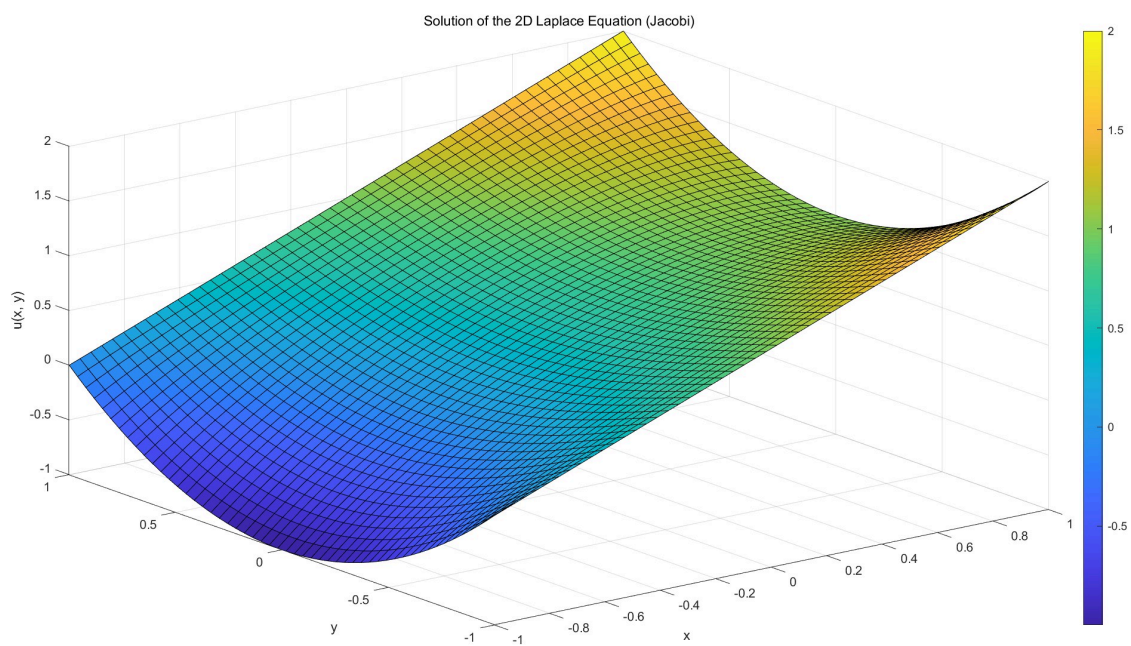
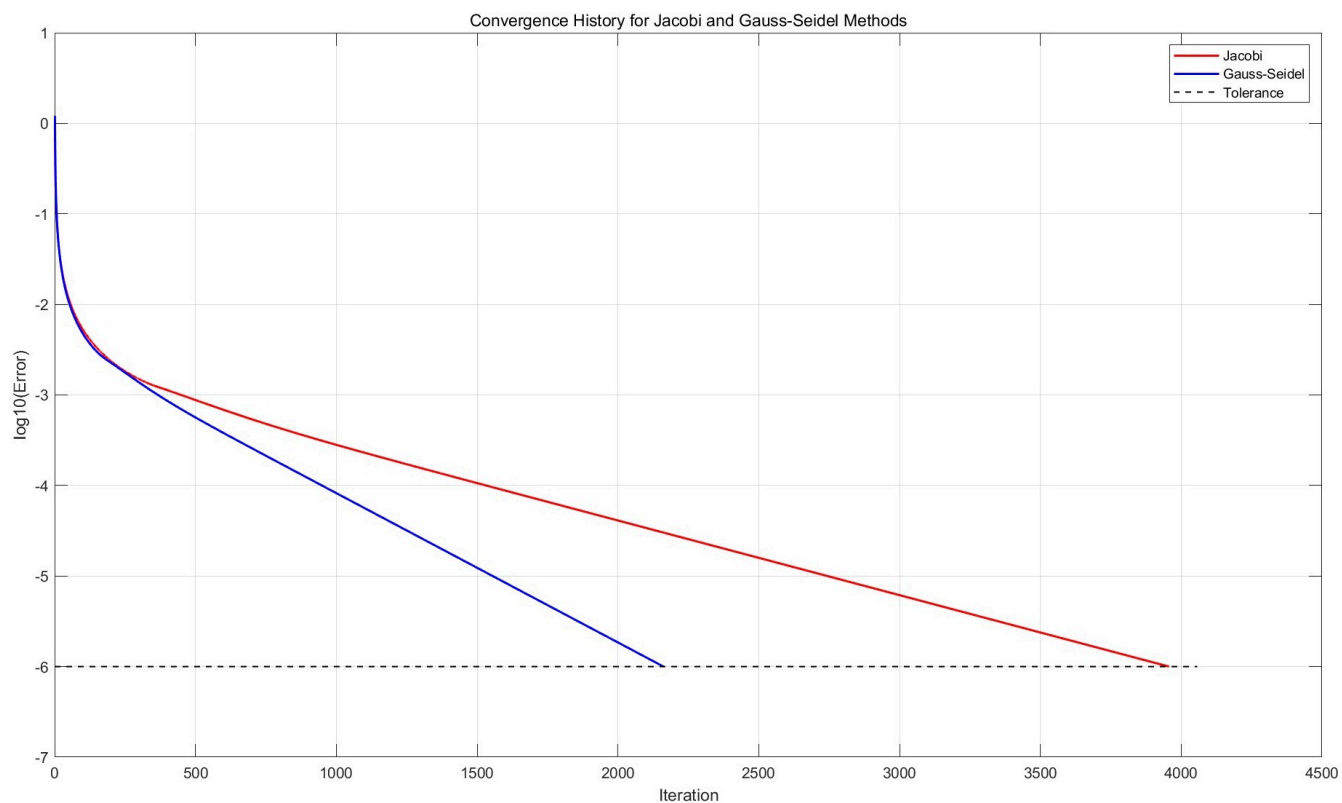
```

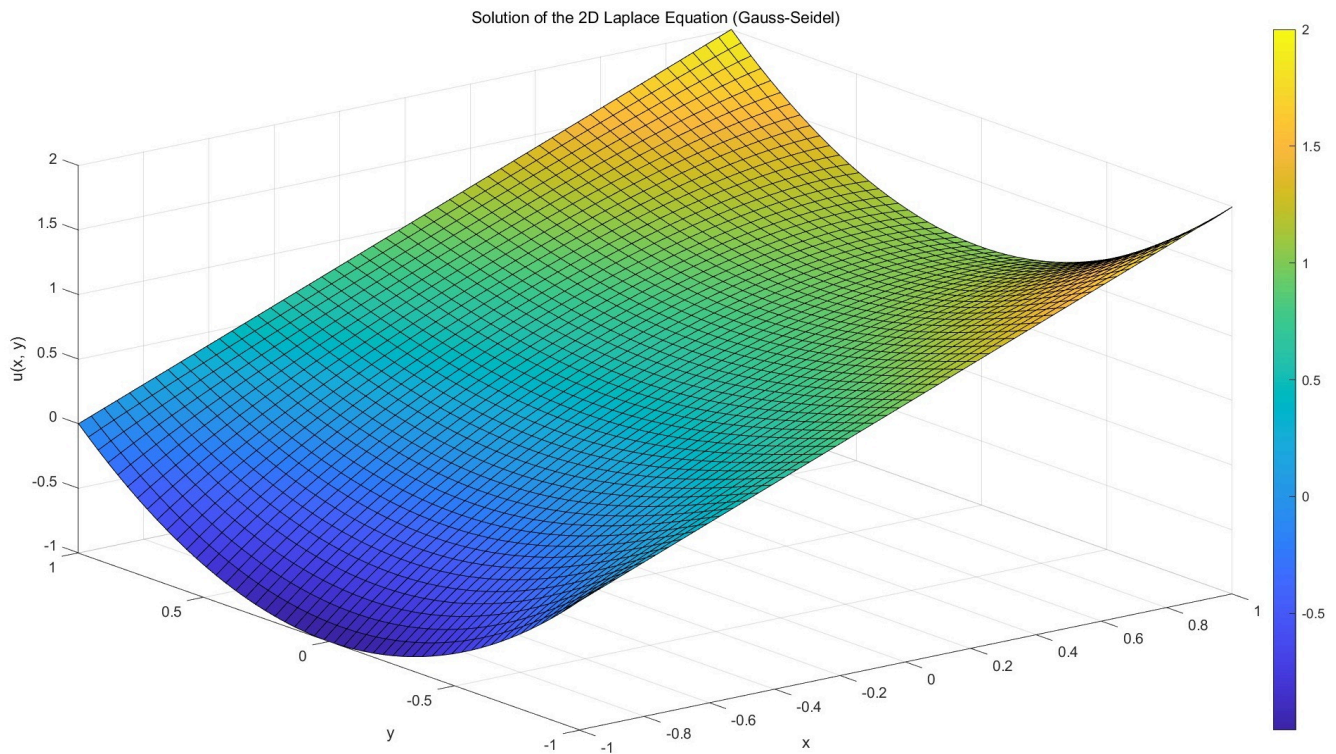
运行结果:

```

Jacobi: Convergence reached after 3957 iterations.
Gauss-Seidel: Convergence reached after 2162 iterations.

```





Problem 4

Solve the partial differential equation

$$\begin{cases} \frac{\partial u(x,t)}{\partial t} = \frac{\partial^2 u(x,t)}{\partial x^2}, & (0 < x < 1) \\ u(x, 0) = 1 - x^2, & (0 < x < 1) \\ u(0, t) = 1 - 2t, \quad u(1, t) = -2t, & (0 \leq t \leq 1) \end{cases}$$

with different finite difference schemes.

Observe the convergence and error propagation using a few different step sizes.

(1) 精确解

容易验证精确解为 $u(x, t) = 1 - 2t - x^2$

我们将 $[0, 1]$ 位置区间均匀划分成 $n_x + 1$ 段, 也就是在内部有 n_x 个"未知节点", $\Delta x = 1/(n_x + 1)$

我们将 $[0, 1]$ 时间区间均匀划分为 n_t 段, $\Delta t = 1/n_t$

一般取 $r = \Delta t/(\Delta x)^2 \leq 1/2$

记 $u_i^n := u(x_i, t_n)$

根据边界条件我们有 $u_0^n = u(0, t_n) = 1 - 2t_n$, $u_{n_x+1}^n = u(1, t_n) = -2t_n$.

Matlab 代码为:

```
% 参数设置
nx = 29;           % 空间内部节点数
nt_ftcs = 1800;    % FTCS时间步数, 满足r=0.5
nt_btcs = 900;     % BTCS时间步数, r=1
nt_cn = 900;       % CN时间步数, r=1

dx = 1/(nx+1);    % 空间步长
dt_ftcs = 0.5*dx^2; % FTCS时间步长
```

```

dt_btcs = dx^2;      % BTCS时间步长
dt_cn = dx^2;        % CN时间步长

x = 0:dx:1;         % 空间网格
t_ftcs = 0:dt_ftcs:1;
t_btcs = 0:dt_btcs:1;
t_cn = 0:dt_cn:1;

% 精确解函数
exact_sol = @(x,t) 1 - 2*t - x.^2;

```

(2) FTCS

FTCS (Forward-Time Centered-Space) 显式格式:

- 时间导数 $\partial u / \partial t$ 在时刻 t_n 处用向前差分近似:

$$\left. \frac{\partial u}{\partial t} \right|_{(x_i, t_n)} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}$$

- 空间二阶导数 $\partial^2 u / \partial x^2$ 在节点 x_i , 时刻 t_n 处, 用 t_n 处的中心差分近似:

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{(x_i, t_n)} \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$$

于是我们有:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} \quad (i = 1, \dots, n_x)$$

$$\Downarrow$$

$$u_i^{n+1} = u_i^n + r(u_{i+1}^n - 2u_i^n + u_{i-1}^n) \quad (i = 1, \dots, n_x)$$

$$\text{where } r = \frac{\Delta t}{(\Delta x)^2}$$

Matlab 代码:

```

% FTCS显式方法
u_ftcs = zeros(length(x), length(t_ftcs));
u_ftcs(:,1) = exact_sol(x, 0); % 初始条件

for n = 1:length(t_ftcs)-1
    u_next = u_ftcs(:,n);
    u_next(1) = 1 - 2*t_ftcs(n+1); % 左边界
    u_next(end) = -2*t_ftcs(n+1); % 右边界
    for i = 2:length(x)-1
        u_next(i) = u_ftcs(i,n) + (dt_ftcs/dx^2)*(u_ftcs(i+1,n) - 2*u_ftcs(i,n) + u_ftcs(i-1,n));
    end
    u_ftcs(:,n+1) = u_next;
end

```

(3) BTCS

BTCS (Backward-Time Centered-Space) 隐式格式:

- 时间导数 $\partial u / \partial t$ 在时刻 t_n 处用向前差分近似:

$$\left. \frac{\partial u}{\partial t} \right|_{(x_i, t_n)} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}$$

- 空间二阶导数 $\partial^2 u / \partial x^2$ 在节点 x_i , 时刻 t_n 处, 用 t_{n+1} 处的中心差分近似:

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{(x_i, t_n)} \approx \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2}$$

于是我们有:

$$\begin{aligned} \frac{u_i^{n+1} - u_i^n}{\Delta t} &= \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} \quad (i = 1, \dots, n_x) \\ &\Downarrow \\ -ru_{i-1}^{n+1} + (1 + 2r)u_i^{n+1} - ru_{i+1}^{n+1} &= u_i^n \quad (i = 1, \dots, n_x) \\ \text{where } r &= \frac{\Delta t}{(\Delta x)^2} \end{aligned}$$

其矩阵形式为:

$$\begin{bmatrix} 1+2r & -r & & & \\ -r & 1+2r & -r & & \\ & \ddots & \ddots & \ddots & \\ & & -r & 1+2r & -r \\ & & & -r & 1+2r \end{bmatrix} \begin{bmatrix} u_1^{n+1} \\ u_2^{n+1} \\ \vdots \\ u_{n_x-1}^{n+1} \\ u_{n_x}^{n+1} \end{bmatrix} = \begin{bmatrix} u_1^n + ru_0^{n+1} \\ u_2^n \\ \vdots \\ u_{n_x-1}^n \\ u_{n_x}^n + ru_{n_x+1}^{n+1} \end{bmatrix}$$

其中 $ru_0^{n+1}, ru_{n_x+1}^{n+1}$ 是来自左端点、右端点的边界值.

Matlab 代码为:

```
% BTCS隐式方法
u_btcs = zeros(length(x), length(t_btcs));
u_btcs(:,1) = exact_sol(x, 0);

% 构造系数矩阵
r_btcs = dt_btcs/dx^2;
A_btcs = gallery('tridiag', nx, -r_btcs, 1+2*r_btcs, -r_btcs);

for n = 1:length(t_btcs)-1
    d = u_btcs(2:end-1, n);
    d(1) = d(1) + r_btcs*(1 - 2*t_btcs(n+1)); % 左边界
    d(end) = d(end) + r_btcs*(-2*t_btcs(n+1)); % 右边界

    u_internal = A_btcs \ d;
    u_btcs(:,n+1) = [1-2*t_btcs(n+1); u_internal; -2*t_btcs(n+1)];
```

(4) CN

CN (Crank-Nicolson) 格式:

- 时间导数 $\partial u / \partial t$ 在时刻 t_n 处用向前差分近似:

$$\left. \frac{\partial u}{\partial t} \right|_{(x_i, t_n)} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}$$

- 空间二阶导数 $\partial^2 u / \partial x^2$ 在节点 x_i , 时刻 t_n 处, 用 t_n, t_{n+1} 处的中心差分的平均值来近似:

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{(x_i, t_n)} \approx \frac{1}{2} \left[\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} + \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} \right]$$

于是我们有:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{1}{2} \left[\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} + \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} \right] \quad (i = 1, \dots, n_x)$$

$\Downarrow$

$$u_i^{n+1} - u_i^n = \frac{r}{2} \left[\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2} + \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{(\Delta x)^2} \right] \quad (i = 1, \dots, n_x)$$

$$\text{where } r = \frac{\Delta t}{(\Delta x)^2}$$

$\Downarrow$

$$\left[-\frac{r}{2} u_{i-1}^{n+1} + (1+r) u_i^{n+1} - \frac{r}{2} u_{i+1}^{n+1} \right] = \left[-\frac{r}{2} u_{i-1}^n + (1-r) u_i^n + \frac{r}{2} u_{i+1}^n \right] \quad (i = 1, \dots, n_x)$$

其矩阵形式为:

$$\begin{aligned} Au^{(n+1)} &= \begin{bmatrix} 1+2r & -r & & & \\ -r & 1+2r & -r & & \\ & \ddots & \ddots & \ddots & \\ & & -r & 1+2r & -r \\ & & & -r & 1+2r \end{bmatrix} \begin{bmatrix} u_1^{n+1} \\ u_2^{n+1} \\ \vdots \\ u_{n_x}^{n+1} \\ u_{n_x+1}^{n+1} \end{bmatrix} \\ &= \begin{bmatrix} 1-2r & r & & & \\ r & 1-2r & r & & \\ & \ddots & \ddots & \ddots & \\ & & r & 1-2r & r \\ & & & r & 1-2r \end{bmatrix} \begin{bmatrix} u_1^n \\ u_2^n \\ \vdots \\ u_{n_x}^n \\ u_{n_x+1}^n \end{bmatrix} + r \begin{bmatrix} u_0^n + u_0^{n+1} \\ 0 \\ \vdots \\ 0 \\ u_{n_x}^n + u_{n_x+1}^{n+1} \end{bmatrix} \\ &= Bu^{(n)} + rb^{(n)} \end{aligned}$$

Matlab 代码为:

```
% Crank-Nicolson方法
```

```

u_cn = zeros(length(x), length(t_cn));
u_cn(:,1) = exact_sol(x, 0);

r_cn = dt_cn/dx^2;
A_cn = gallery('tridiag', nx, -r_cn/2, 1+r_cn, -r_cn/2);
B_cn = gallery('tridiag', nx, r_cn/2, 1-r_cn, r_cn/2);

for n = 1:length(t_cn)-1
    u0_n = 1 - 2*t_cn(n);
    u0_next = 1 - 2*t_cn(n+1);
    u1_n = -2*t_cn(n);
    u1_next = -2*t_cn(n+1);

    rhs = B_cn * u_cn(2:end-1, n);
    rhs(1) = rhs(1) + (r_cn/2)*(u0_n + u0_next);
    rhs(end) = rhs(end) + (r_cn/2)*(u1_n + u1_next);

    u_internal = A_cn \ rhs;
    u_cn(:,n+1) = [u0_next; u_internal; u1_next];
end

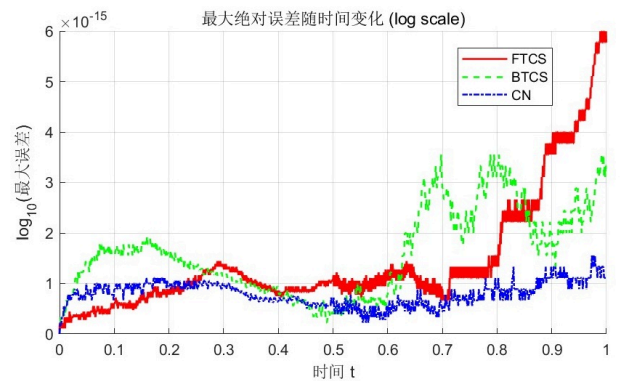
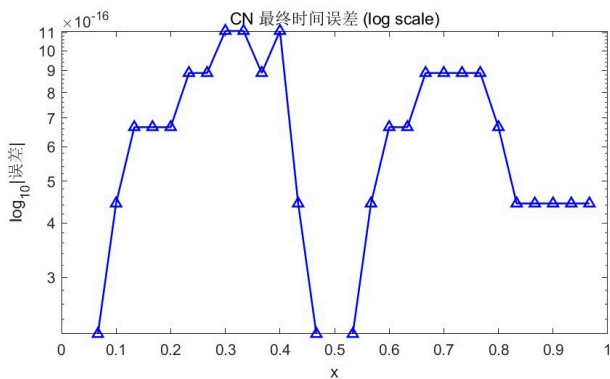
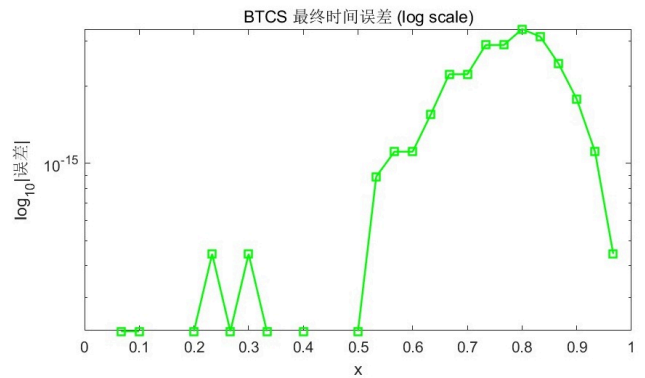
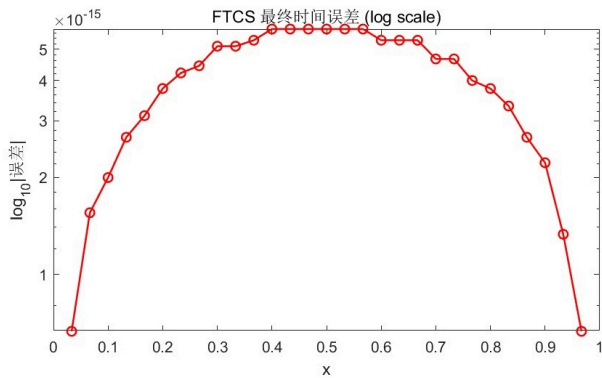
```

(5) 运行结果

FTCS最大误差: 5.7732e-15

BTCS最大误差: 3.3307e-15

CN最大误差: 1.1102e-15



Problem 5 (optional)

Use the Galerkin method to solve the Fredholm integral equation

$$u(t) - \int_0^1 (2t + 3)s^t u(s) ds = t^{1/2} - 2$$

Compare your solution with the exact one ($u(t) = t^{1/2}$).